

Chapitre 4

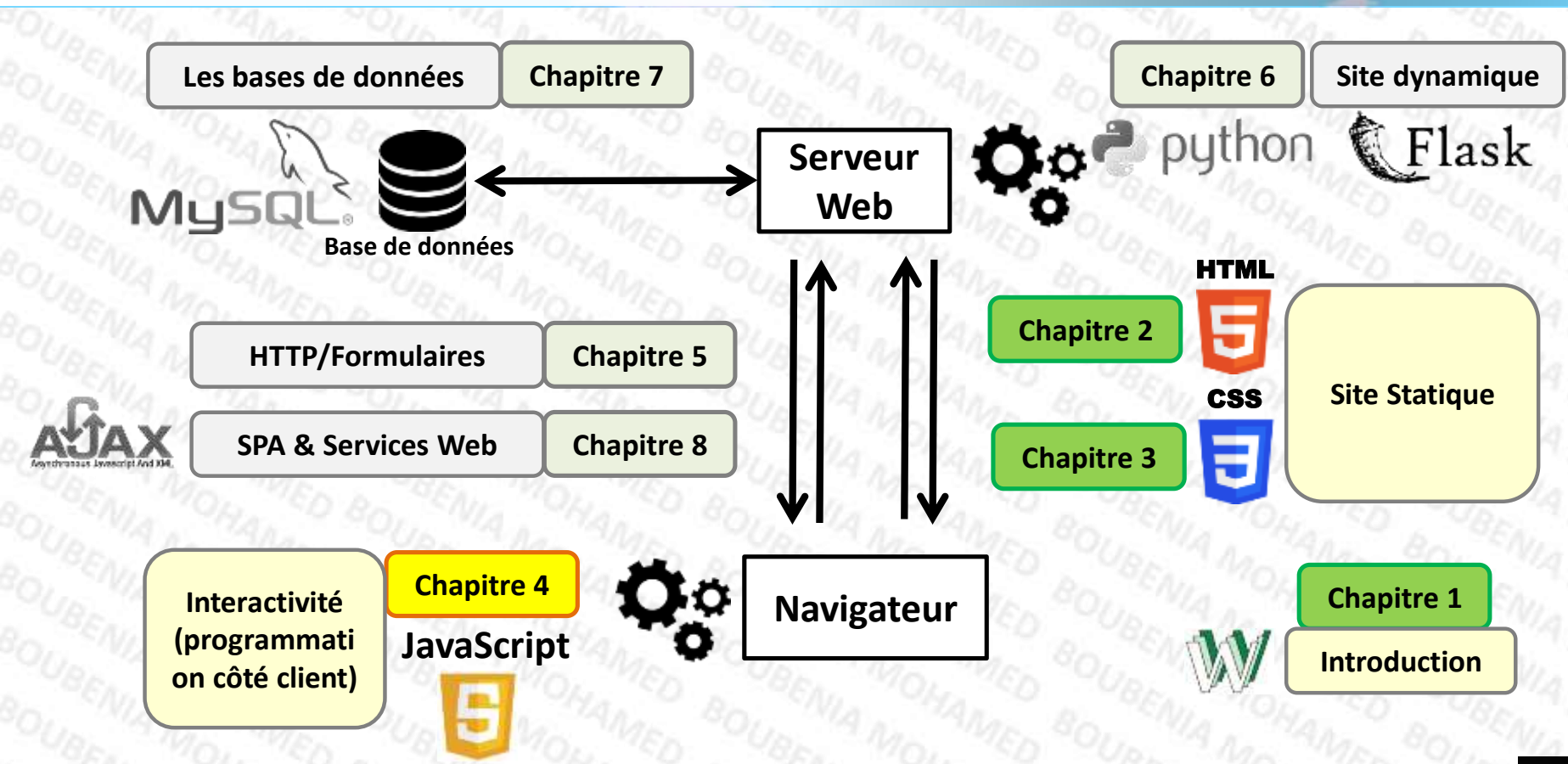
Site Interactif

JavaScript

Dr. BOUBENIA Mohamed
mo.boubenia@gmail.com

25/10/2025

Où sommes nous ?



Plan de cette séance

1. Rappel
2. Les Bases de la programmation côté client
3. Les Bases du Langage JavaScript
4. Manipulation du DOM
5. Le BOM (Browser Object Model)
6. Les Evénements

BOUBENIA MOHAMED

1
Rappel

BOUBENIA MOHAMED

1- Rappel

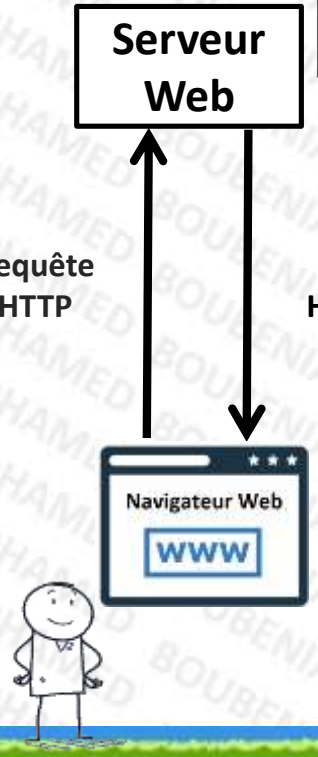
Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques

Les langage HTML et CSS suffisent pour créer des sites Web statiques :

- CV en ligne
- Site vitrine ou Portfolio (Galerie)
- Blog Statique
- Site de présentation d'un organisme
- Landing page (page de destination)
- Site de documentation technique
- Site événementiel
- Site de recette





Suffisant pour la présentation

HTML CSS

1- Rappel

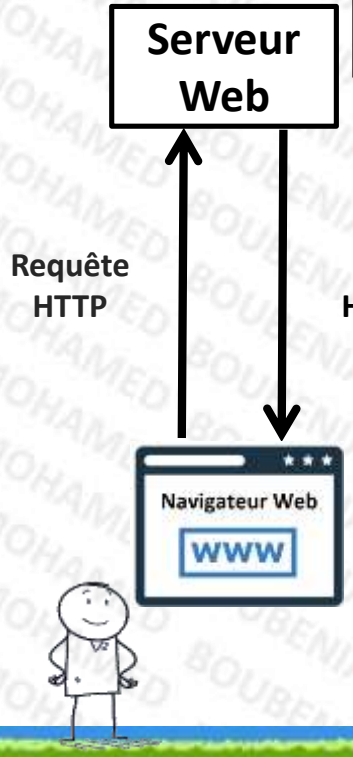
Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques

Les langage HTML et CSS suffisent pour créer des sites Web statiques :

- CV en ligne
- Site vitrine ou Portfolio (Galerie)
- Blog Statique
- Site de présentation d'un organisme
- Landing page (page de destination)
- Site de documentation technique
- Site événementiel
- Site de recette





Suffisant pour la présentation

HTML CSS

…Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

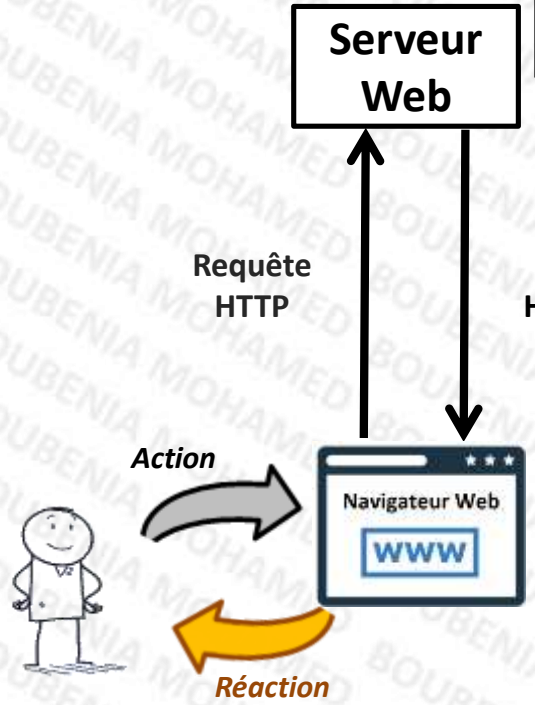
Sites Statiques **Interactif**

Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.





Suffisant pour la présentation

HTML CSS

…Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques **Interactif**

Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

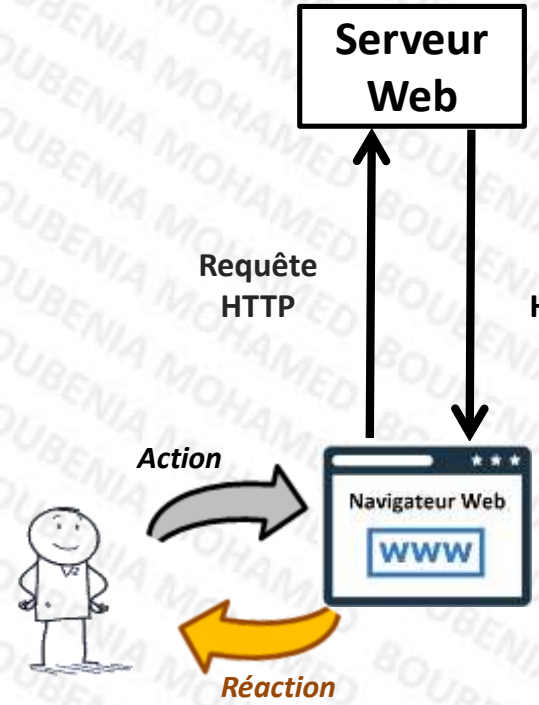
- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés



Suffisant pour la présentation

HTML CSS



…Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?

1- Rappel



Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques | Interactif

Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bou

- Une élément HTML s'affiche/se modifie/se m
- ou Une propriété CSS chang
- ou Un résultat d'un calcul est
- ou Suivant la véracité d'une c contenu de la page Web chan
- ou Vérification du contenu d' de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés

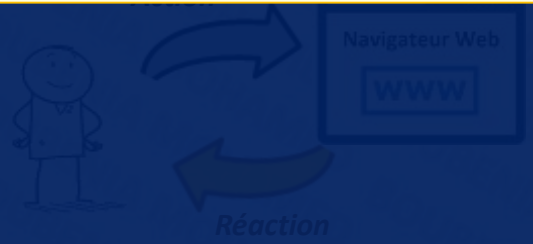
Serveur

Suffisant pour la présentation

HTML CSS

...Mais, à eux seuls, ils ne permettent pas l'interactivité avec l'utilisateur!

De quelle interactivité parle-ton ?



Mais rappelez-vous

HTML n'est pas un langage de programmation

Un langage de programmation permet de donner des instructions la machine, en utilisant des variables, conditions, boucles, fonctions, etc.
HTML/CSS ne sont que des langages déclaratifs

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques **Interactif**

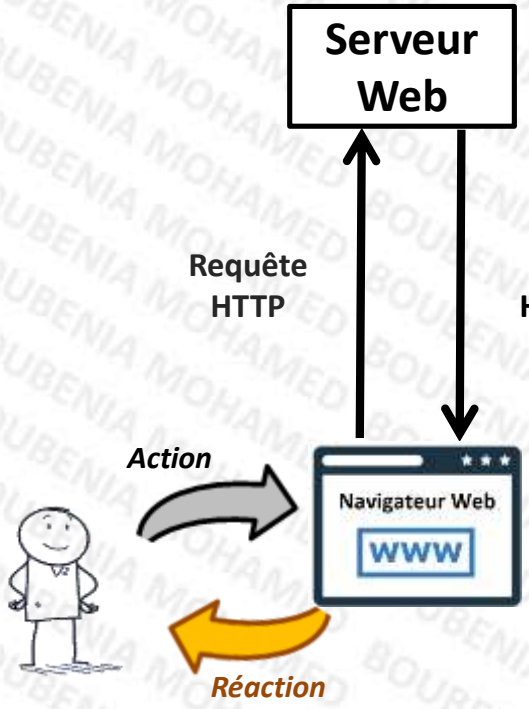
Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés





Suffisant pour la présentation

HTML CSS

JavaScript

Cette programmation se fait dans des scripts en langage JavaScript

1- Rappel

Quels types de site pourrait-on créer avec HTML et CSS seulement ?

Sites Statiques **Interactif**

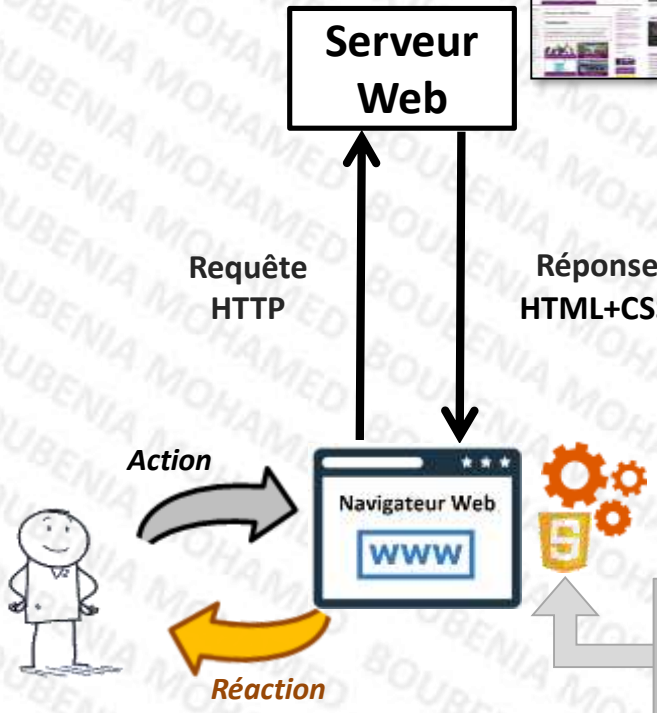
Exemples d'interaction:

Suite à un événement provoqué par l'utilisateur (ex. clique sur un bouton):

- Une élément HTML s'affiche/se modifie/se masque.
- ou Une propriété CSS change.
- ou Un résultat d'un calcul est affiché.
- ou Suivant la véracité d'une condition le contenu de la page Web change.
- ou Vérification du contenu d'un champs de saisie.
- etc.

Les comportements que l'on souhaite prévoir sur notre page Web doivent être programmés





Suffisant pour la présentation

HTML CSS

JavaScript

Ces scripts sont exécutés par le navigateur au besoin (ex. un événement click se produit)

2
Les Bases de la programmation côté client

BOUBENIA MOHAMED

2- Les Bases de la programmation côté client

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>Incrémenter</button>
    </body>
</html>
```

1 Prenons l'exemple suivant. Nous avons donc un code HTML qui contient:

2 un élément H1 "Compteur=0"

3 et un élément Button (bouton) "Incrémenter"



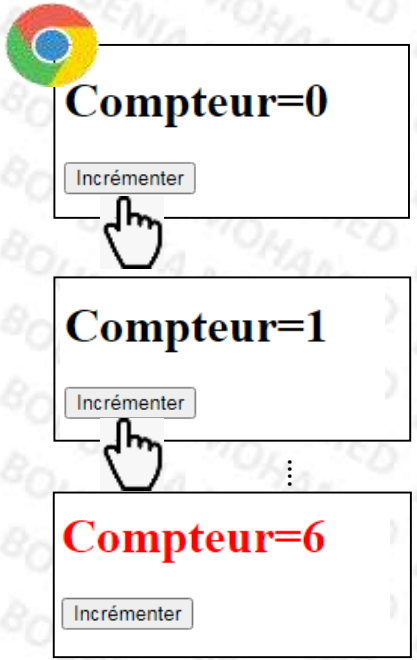
2- Les Bases de la programmation côté client

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button>Incrémenter</button>
    </body>
</html>
```



2- Les Bases de la programmation côté client

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

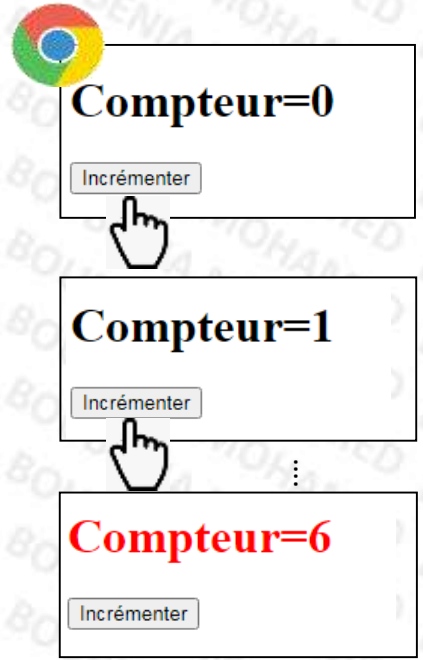
```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Ce comportement doit être programmé dans un script JS (JavaScript).

2- Les Bases de la programmation côté client

On voudrait prévoir le comportement suivant

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

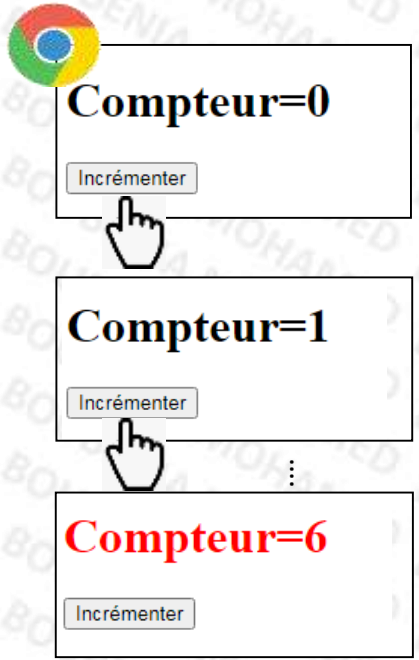
Ce scripts sera exécuté par le navigateur lors d'un click sur le bouton

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Ce comportement doit être programmé dans un script JS (JavaScript).

2- Les Bases de la programmation côté client

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

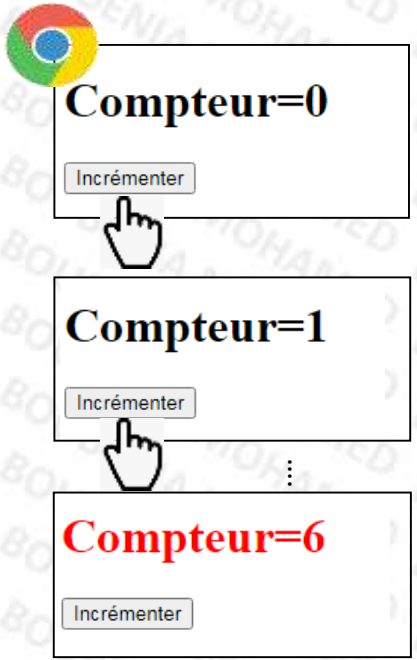
Décortiquons un peu ce comportement

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



2- Les Bases de la programmation côté client

Décortiquons un peu ce comportement

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, et si on dépasse 5, il devient rouge.

Evénement

lorsque l'utilisateur clique sur le bouton "Incrémenter",

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

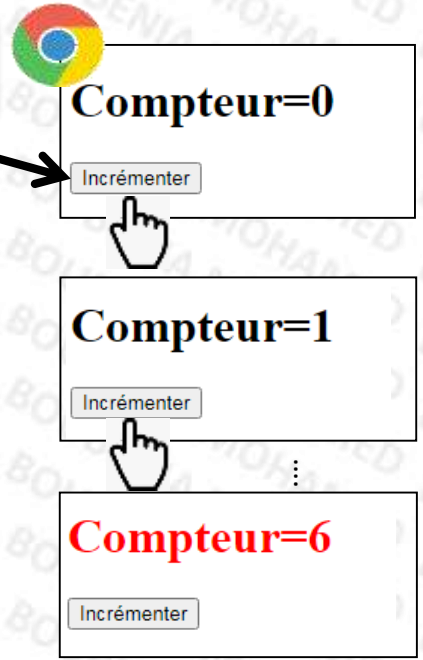
Nous avons attaché à l'élément bouton un événement (au Clic). Ca veut dire, quand il sera cliqué

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



2- Les Bases de la programmation côté client

Décortiquons un peu ce comportement

lorsque l'utilisateur clique sur le bouton "Incrémenter", **le compteur s'incrémente de 1**, et si on dépasse 5, il devient rouge.

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

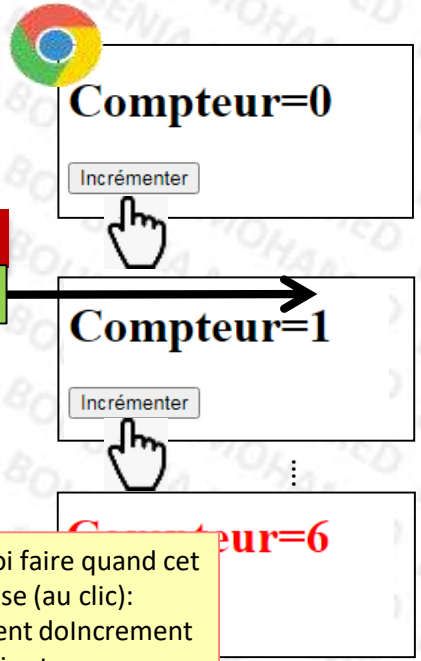
Evénement

lorsque l'utilisateur clique sur le bouton "Incrémenter",

Traitement

le compteur s'incrémente de 1

Changement visuel



Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```

Nous précisons quoi faire quand cet événement se réalise (au clic):
Ici, c'est le traitement doIncrement qui est exécuté (qui est une fonction).

Puis un changement visuel est effectué, qui consiste à modifier le contenu de l'élément h1

2- Les Bases de la programmation côté client

Décortiquons un peu ce comportement

lorsque l'utilisateur clique sur le bouton "Incrémenter", le compteur s'incrémente de 1, **et si on dépasse 5, il devient rouge.**

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

lorsque l'utilisateur clique sur le bouton "Incrémenter",

Traitement

le compteur s'incrémente de 1

Changement visuel

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

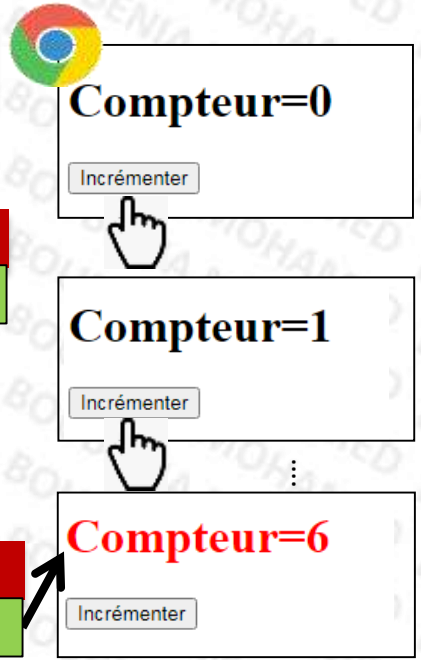
    /*Prévoir le comportement à faire quand l'événement C      aché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```

Traitement

et si on dépasse 5, il devient rouge.

Changement visuel



Nous pouvons aussi mettre une condition comme un traitement, si elle est vérifiée, on fait un changement visuel. Ici, on a changé la couleur de l'élément h1

2- Les Bases de la programmation côté client

Résumons

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

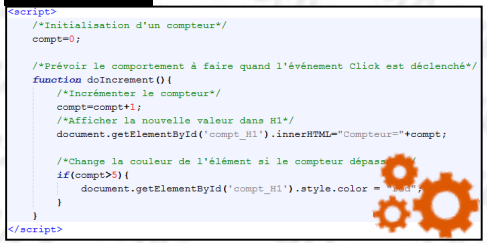
Evénement

Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

2- Les Bases de la programmation côté client

Résumons

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

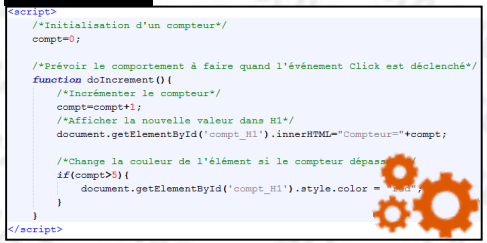
Evénement

Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation

Oui… Mais où mettre ces scripts? **Il existe 3 méthodes**

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 1: Enligne

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt H1">Compteur=0</h1>
        <button onclick="document.getElementById('compt_H1').innerHTML='Hello!!!'">
            Incrémenter
        </button>
    </body>
</html>
```

Consiste à mettre le script directement dans la balise ouvrante de l'élément concerné, en précisant l'événement.
Idéal quand il s'agit d'un petit traitement
(peu recommandé)

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 1: Enligne

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="document.getElementById('compt_H1').innerHTML='Hello!!!'">
            Incrémenter
        </button>
    </body>
</html>
```

Méthode 2:Dans un élément <script></script> à la fin de body

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
        <script>
            /*Traitement*/
            function doIncrement(){
                /* do Increment*/
            }
        </script>
    </body>
</html>
```

Consiste à mettre le script à l'intérieur d'un élément <script></script>, et de mettre cet élément n'importe où dans le document HTML (dans le body, où dans le head)

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 1: Enligne

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="document.getElementById('compt_H1').innerHTML='Hello!!!'">
            Incrémenter
        </button>
    </body>
</html>
```

Méthode 2:Dans un élément <script></script> à la fin de body

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
        <script>
            /*Traitement*/
            function doIncrement(){
                /* do Increment*/
            }
        </script>
    </body>
</html>
```

Consiste à mettre le script dans un autre fichier qui porte l'extension .js

Dans cet exemple, nous avons mis notre script dans « mon_script.js"

et nous lui avons fait référence dans notre fichier HTML, grâce à l'élément script, dans l'élément Head.

Vous avez la possibilité de charger plusieurs fichiers

Méthode 3: Fichier JS

```html
<!DOCTYPE html>
<html>
    <head>
        <script src="mon_script.js"></script>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

mon_script.js

```js
/*Traitement*/
function doIncrement(){
    /* do Increment*/
}
```

2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Méthode 3: Fichier JS avec URL

```html
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
      <!-- Script externe chargé depuis une URL -->
      <script src="https://cdnjs.cloudflare.com/ajax/libs/moment.js/2.30.1/moment.min.js"></script>
      <script>
        console.log("Date actuelle :", moment().format());
      </script>
    </body>
</html>
```

Vous avez également la possibilité de faire référence à des fichiers JS sur le Web (ex. bibliothèques)

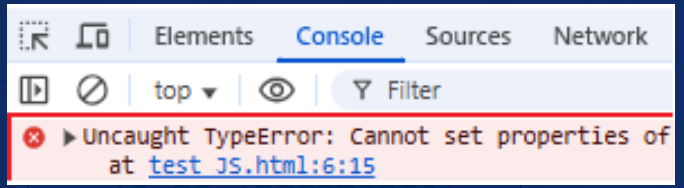
Erreur fréquente de débutant

Lorsque vous mettez votre script dans le head, il sera exécuté avant le chargement en entier du DOM (votre page)

Méthode 3: Fichier JS

```
<!DOCTYPE html>
<html>
    <head>
        <script src="script.js">
            /*Changer la couleur de H1*/
        </script>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
    </body>
</html>
```

Ce code produira une erreur, car le code JS essayera de manipuler un élément (h1) qui n'a pas encore été créé.



2- Les Bases de la programmation côté client

Où mettre les Scripts ?

Solutions possibles

1

Vous pouvez différer l'exécution de script.js après le chargement de la page.

Méthode 3: Fichier JS

```
<!DOCTYPE html>
<html>
    <head>
        <script src="script.js" defer>
            /*Changer la couleur de H1*/
        </script>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
    </body>
</html>
```

2

Vous pouvez explicitement mettre l'élément script à la fin de body

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <script src="script.js" defer></script>
    </body>
</html>
```

Vous pouvez également utiliser un événement approprié **DOMContentLoaded**

2- Les Bases de la programmation côté client

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

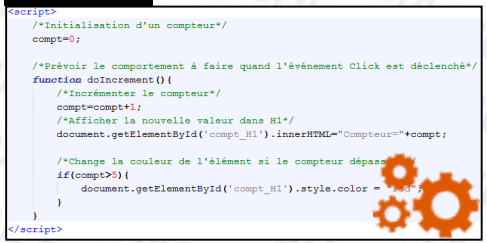
Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Nous allons décortiquer

Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.

2- Les Bases de la programmation côté client

Les bases du langage JavaScript

D'abord, Il est important de connaitre le langage JavaScript (syntaxe, sémantique, etc.)

Nous allons décortiquer

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation

3- Les Bases du Langage JavaScript

C'est quoi ?

JavaScript est principalement un langage de programmation côté client.

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépasse 5*/
        if(compt>5){
            document.getElementById('compt_H1').style.color = "red";
        }
    }
</script>
```



Historique

- JavaScript (anciennement ECMAScript) a été créé par Netscape (le fameux navigateur Web),en 1995.
- **But:** Permettre aux développeurs Web d'incorporer un code exécutable dans leurs pages Web.
- **Effet:** Une page Web pouvait donc contenir un script en JavaScript, qui est exécuté quand l'utilisateur visite la page (interactivité ou traitement).

3- Les Bases du Langage JavaScript

Caractéristiques

- **Langage multi-paradigmes**
  Impérative, Fonctionnelle, POO.
- **Syntaxe**
  Fortement inspirée de C++ et Java.
  Ne pas confondre avec Java (aucun lien, même si similitude dans le nom).

…Mais, Différences majeures avec des langages comme C++ et Java ! :

- **Langage non compilé, mais interprété:**
  Il n'y a pas d'étape de compilation. A la place, un interpréteur (présent dans le navigateur (ex. V8, SpiderMonkey, etc.)), lit le code JS, interprète chaque ligne, et l'exécute (compilation Just-In-Time (JIT)).
- **Langage dynamique et faiblement typé**
  Une variable peut à tout moment changer de type. JS convertit automatiquement les types si nécessaire (ex. 2+ "2" donne "22").
- **Garbage Collector**
  Intégré dans le moteur JavaScript pour la gestion de la mémoire.
- **Spécification:** ECMAScript (ES) Specification.

Versions ECMAScript

Edition	Nom Officiel	Date publication
ES16	ES2025	June 2025
…	…	…
ES10	ES2019	Summer 2019
ES9	ES2018	June 2018
ES8	ES2017	June 2017
ES7	ES2016	June 2016
ES6	ES2015	June 2015
ES5.1	ES5.1	June 2011
ES5	ES5	December 2009
ES4	ES4	Abandoned
ES3	ES3	December 1999
ES2	ES2	June 1998
ES1	ES1	June 1997

3- Les Bases du Langage JavaScript

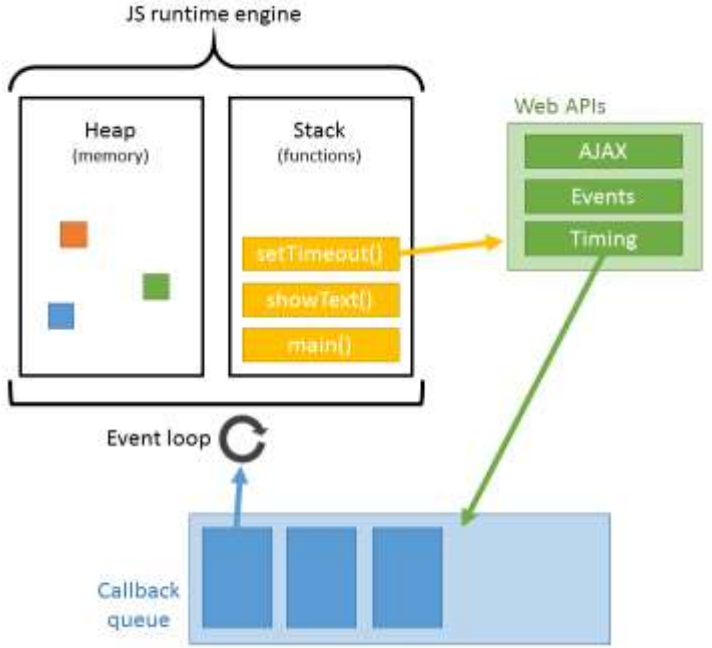
Caractéristiques

Single Threaded (Une seule file d'exécution) avec Event-Loop

JS s'exécute sur un seul thread principal. Cela signifie qu'il ne peut pas exécuter plusieurs instructions à la fois (pas de multithreading natif). Mais, il n'est pas bloquant grâce à son modèle asynchrone

4 composant principaux:

1. **Event Loop**
2. **Stack (pile d'appels): Syncrhone**
3. **Heap (tas de mémoire):**
4. **Callback Queue (file): Asynchrone**



Déroulement:

1. L'Event Loop (la boucle d'événements, sur le thread principal) exécute le code JavaScript ligne par ligne dans la Call Stack.
2. Les objets et variables sont stockés dans la Heap memory.
3. Lorsqu'une opération asynchrone est rencontrée (par ex. setTimeout, appel à une API, événement DOM), elle est déléguée à l'environnement du navigateur (les Web APIs).
4. Quand cette opération est terminée, son callback est placé dans la Callback Queue (ou Task Queue).
5. L'Event Loop surveille en permanence la Call Stack : dès qu'elle est vide, il y récupère la prochaine tâche de la Callback Queue pour l'exécuter.

3- Les Bases du Langage JavaScript

Familiarités avec les langages de programmation

Contrairement au langage C ou Java, en JavaScript il n'y a pas de notion de fonction principale (la fameuse fonction main()).

A la place, le navigateur interprète et exécute le code de haut vers le bas, et compris les fichiers JS externes.

Comme avec tout langage de programmation, en JavaScript, il y a aussi la console qu'on peut utiliser pour afficher (on écrit: console.log("text à afficher")).

Le console est accessible dans n'importe quel navigateur. Par exemple, ici, on a ouvert la page maPage.html, puis les outils de développement, puis l'onglet Console.

Recommandé lorsqu'on fait des teste (ex. les valeurs des variables).

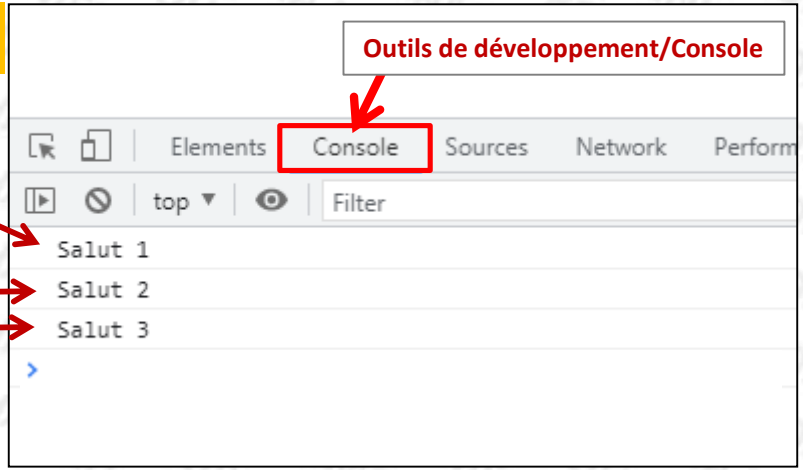
maPage.html

```
<!DOCTYPE html>
<html>
    <head>
        <script src="mon_script.js"></script>
    </head>
    <body>
        <h1>Titre1</h1>
        <script>
            console.log("Salut 2");
            console.log("Salut 3");
        </script>
    </body>
</html>
```

Mon_script.js

```
console.log("Salut 1");
```

Console.log()



3- Les Bases du Langage JavaScript

Familiarités avec les langages de programmation

D'autres soties possibles

innerHTML/innerText

```
<p></p>
<script>
    elt = document.getElementsByTagName("p")[0];
    elt.innerHTML="Hello World"
</script>
```

Hello World

Alert()

```
<script>
    alert("Hello World");
</script>
```



3- Les Bases du Langage JavaScript

Noyau du langage JS

1. Syntaxe
2. Variables et Constantes
3. Les opérateurs
4. Les conditions
5. Les boucles
6. Les fonctions
7. Portée d'une variable

BOUBENIA MOHAMED

3- Les Bases du Langage JavaScript

- Syntaxe proche de celle du langage C: quasiment les mêmes opérateurs, les mêmes instructions.
- Toutes les instructions doivent se terminer par un point virgule (;) bien que ça reste facultatif.

```
console.log("Hello World");
```

Les affectations

Signe égale: Ici on a fait un calcul, et le résultat est affecté à une variable "a"

```
a = 1 + 2;
```

Bloc de code

Exemple: Fonction, conditions, etc

```
{
    // Intruction 1
    // Intruction 2
}
```

Les commentaires

// pour chaque ligne

```
// console.log("Hello World");
```

Sur plusieurs lignes : commence par /* et se termine par */

```
/*
    console.log("Hello World");
    console.log("Hello World");
*/
```

3- Les Bases du Langage JavaScript

Noyau du langage JS | 2. Variables et Constantes | Les Types primitifs

String
number
boolean
null

```
// Initialisation des variables
nom = 'JOHN';  // chaine de caractère
prenom = "DOE"; // chaine de caratère
age = 30; // entier
note = 15.5 // réel
estAdulte = true; // booléen
x = null; // valeur nulle
const pi = 3.14;
// Appel des variables
console.log(nom, prenom);  // Affichage multiple
console.log("Nom="+nom+", Prénom="+prenom);  // Affichage élaboré
console.log(`Bonjour ${nom}`); // Affichage formaté (backstick)
console.log(age);
console.log(note);
console.log(estAdulte);  // Affichage multiple
console.log(x);
console.log(pi);
console.log(estadulte);  // Erreur (var non trouvée) car sensibilité à la casse
// l'interpéteur s'arrête à la rencotre d'une erreur
/*
    le code ici sera ignoré
*/
```

> Console

```
JOHN DOE
Nom=JOHN, Prénom=DOE
Bonjour JOHN
30
15.5
true
null
3.14
Uncaught ReferenceError: estadulte is not defined
    at test.html:31
>
```

3- Les Bases du Langage JavaScript

Noyau du langage JS | 2. Variables et Constantes | Les Types Complexes

Les Tableaux

```
//tableau d'entier
monTableau1 = new Array(1,2,3,4,5);
//une autre manière
monTableau2 = [1,2,3,4,5];
console.log(monTableau1);
console.log(monTableau2);

//tableau de chaine de caractère
monTableau3 = ['A','B','test','D'];
console.log(monTableau3);

//tableau avec un contenu divers
monTableau4 = ['A','B',10,true];
console.log(monTableau4);
// Accès au contenu du tablau
console.log(monTableau4[0]);
console.log(monTableau4[3]);

// Modification
monTableau4[0] = 'BOB';
console.log(monTableau4);
```

> Console

```
▸ (5) [1, 2, 3, 4, 5]
▸ (5) [1, 2, 3, 4, 5]
▸ (4) ['A', 'B', 'test', 'D']
▸ (4) ['A', 'B', 10, true]
A
true
▸ (4) ['BOB', 'B', 10, true]
```

Destructuration

```
monTableau1 = [1,2,3,4,5]
const [x,y,z]= monTableau1
// Donne 1, 2, 3
```

Méthodes utiles

```
// ajouter à la fin
monTableau1.push(6);

// Supprime le dernier élément
monTableau1.pop();

// vérifie si le tableau contien un élément
monTableau1.includes(2)
// donne true

// Retourne le premier élément
// qui satisfait une condition
res = monTableau1.find(n => n > 3);
// donne 4

// filter: Sélectionner certains éléments
grands = monTableau1.filter(n => n > 3);
  // donne [4, 5]

// map: Transformer les valeurs
doubles = monTableau1.map(n => n * 2);
// donne [2, 4, 6, 8, 10]

// reduce: Calcul (somme, produit, etc.)
// acc → accumulateur (initialisé à 0 ici)
// val → valeur courante
somme = monTableau1.reduce
                ((acc, val) => acc + val, 0);
// donne 15
```

3- Les Bases du Langage JavaScript

Les Objets

Dans JS pratiquement tout repose sur les objets: les tableaux, les fonctions, même les classes sont en fait des objets.

```
// déclaration de l'objet person
const person = {
    nom : 'JOHN',
    prenom: 'DOE',
    age: 30,
    centresInterets: ['sport', 'films', 'peinture'],
    adresse: {
        cite: '5 Juillet',
        commune: 'Bab-Ezzouar',
        ville: 'Alger'
    }
};
console.log(person);
console.log(person.nom);      // accès à nom
console.log(person["age"]); // accès à age
```

> Console

```
▶ {nom: 'JOHN', prenom: 'DOE', age: 30, centresInterets: Array(3), adresse: {…}}
JOHN
30
>
```

```
// Rajouter une propriété
person.metier="Ingenieur";

// Modifier une propriété
person.nom="BOB";

// Supprimer une propriété
delete person.metier;
```

Destructuration

```
const person = {
    nom: "JOHN",
    prenom: "DOE"
};

const { nom, prenom } = person;
```

3- Les Bases du Langage JavaScript

Tableau d'Objets

```
const etudiants = [
    {
        id: 1,
        nom: 'nom1',
        prenom: 'prenom1'
    },
    {
        id: 2,
        nom: 'nom2',
        prenom: 'prenom2'
    },
    {
        id: 3,
        nom: 'nom3',
        prenom: 'prenom3'
    }
];
console.log(etudiants);
console.log(etudiants[0].nom);

const etudiantJSON = JSON.stringify(etudiants);
console.log(etudiantJSON);
```

```
▼(3) [{…}, {…}, {…}]
  ▶0: {id: 1, nom: 'nom1', prenom: 'prenom1'}
  ▶1: {id: 2, nom: 'nom2', prenom: 'prenom2'}
  ▶2: {id: 3, nom: 'nom3', prenom: 'prenom3'}
    length: 3
  ▶[[Prototype]]: Array(0)
nom1
[{"id":1,"nom":"nom1","prenom":"prenom1"},{"id":2,"nom":"nom2","prenom":"prenom2"},{"id":3,"nom":"nom3","prenom":"prenom3"}]
>
```

Le tableau d'objets peut aussi être transformé en objet JSON (JavaScript Objet Notation), qui est un format léger, et très utilisé pour les échanges client-serveur.
On le reverra dans la partie Site Web Dynamique

3- Les Bases du Langage JavaScript

Arithmétiques

```
val = 1+2; // Addition
val = 2-1; // Soustraction
val = 2*2; // Multiplication
val = 4/2; // Division
val = 4%2; // Reste Division
val++; // Incrémente de 1
val--; // Décrémente de 1
```

Opérateurs combiné

```
x += 3; // x=x+3
x -= 2; // x=x-2
x *= 4; // x=x*4
x /= 2; // x=x/2
x %= 3; // x=x%3
x **= 2;// x=x²
```

Logiques

```
val1=true;
val2=false;

console.log(val1 && val1); // &&: le ET logique
console.log(val1 && val2);
console.log(val1 || val2); // ||: le OU logique
console.log(!val1);        // ! le NON logique
```

```
true
false
true
false
>
```

Rationnels

```
console.log(2>1);
console.log(2<1);
console.log(2<=1);
console.log(2>=1);
console.log(2==1); // égalité
console.log(2!=1); // inégalité
console.log(5=="5"); // true
console.log(5==="5"); // false
```

```
true
false
false
true
false
true
>
```

3- Les Bases du Langage JavaScript


```
val1=1;
val2=2;
val3=1;

// Condition if avec une seule instruction
if(val2>val1) console.log('val2 est supérieure à val1');

// Condition if avec plusierus instructions
if(val2>val1) {
    console.log('Hello World !');
    console.log('val2 est supérieure à val1');
}

// Condition if et else
if(val2>val1)
    console.log('val2 est supérieure à val1');
else
    console.log('val2 est inférieure à val1');


// Condition if et else if
if(val1>val3)
    console.log('val1 est supérieure à val3');
else if(val1<val3)
    console.log('val1 est inférieure à val3');
else
    console.log('val1 est égale à val3');
```

```
val2 est supérieure à val1
Hello World !
val2 est supérieure à val1
val2 est supérieure à val1
val1 est égale à val3
>
```

```
// Notation Ternaire
val1=1;
(val1==1)? res=true:res=false;
// true
```

3- Les Bases du Langage JavaScript


```
jour = 3;

switch (jour) {
  case 1:
    nomJour = "Lundi";
    break;
  case 2:
    nomJour = "Mardi";
    break;
  case 3:
    nomJour = "Mercredi";
    break;
  default:
    nomJour = "Jour inconnu";
}

console.log(nomJour);
// Mercredi
```

BOUBENIA MOHAMED

3- Les Bases du Langage JavaScript


```
// tableau d'étudiants
const etudiants = [
    {
        id: 1,
        nom: 'nom1',
        prenom: 'prenom1'
    },
    {
        id: 2,
        nom: 'nom2',
        prenom: 'prenom2'
    },
    {
        id: 3,
        nom: 'nom3',
        prenom: 'prenom3'
    }
];
```

```
// Boucle: for
for(let i=0; i<=2; i++){
    console.log(etudiants[i].nom);
}

// Boucle: While
i=0;
while(i<etudiants.length){
    console.log(etudiants[i].nom);
    i++;
}

// Boucle: for..of
for(etudiant of etudiants){
    console.log(etudiant.nom);
}

// Boucle forEach
etudiants.forEach(function(etudiant){
    console.log(etudiant.nom);
});
```

```
nom1
nom2
nom3
>
```

forEach ne peut pas être interrompue avec **break** (utilisé pour interrompre une boucle), et ne peut pas sauter une iteration avec **continue**. Elle ne peut pas faire des **return** non plus.

3- Les Bases du Langage JavaScript

Noyau du langage JS | 6. Les Fonctions

```
// Déclaration d'une fonction sans paramètres
function doCalcul0(){
    console.log(5*10);
}
// Déclaration d'une fonction avec paramètres
function doCalcul1(val1, val2){
    console.log(val1+val2);
}
// Déclaration d'une fonction avec un retour
function doCalcul2(val1, val2){
    return val1*val2;
}

// Appel
doCalcul0();
doCalcul1(1,2);
console.log(doCalcul2(1,2));
```

```
50
3
2
>
```

```
const etudiants = [
    {
        id: 1,
        nom: 'nom1',
        prenom: 'prenom1'
    },
    {
        id: 2,
        nom: 'nom2',
        prenom: 'prenom2'
    },
    {
        id: 3,
        nom: 'nom3',
        prenom: 'prenom3'
    }
];

// Le passage de paramètre se fait par référence
// et non pas par valeur
function doCalcul3(objetEtudiant){
    // modifiera etudiants[1]
    objetEtudiant.id=objetEtudiant.id*10;
}
doCalcul3(etudiants[1]);
console.log(etudiants[1].id);
```

```
20
>
```

3- Les Bases du Langage JavaScript

Noyau du langage JS | 6. Les Fonctions

Définir une valeur par défaut pour le paramètre

```
function doCalcul3(val=1){
    return val*3
}

console.log(doCalcul3());
// Affiche 3
```

- Hoistée
- Propre this
- Réutilisable

Fonction anonyme et invoquée immédiatement

```
(function(val=1) {
  console.log(val*3);
})();
// Affiche 3
```

- Non Hoistée
- Propre this
- Non Réutilisable
- Invocation immédiate
- Protège le code

Fonction fléchée (Arrow Function)

```
doCalcul3 = (val=1) =>{
    return val*3
}

console.log(doCalcul3());
// Affiche 3
```

- Non Hoistée
- This du parent
- Réutilisable
- Utile pour les callbacks

3- Les Bases du Langage JavaScript

Un callback est une fonction passée en paramètre à une autre fonction, et appelée plus tard.

```
function processData(data, callback) {
  console.log("Traitement des données...");
  callback(data);
}

processData("Bonjour", function(result) {
  console.log("Résultat :", result);
});
```

```
function processData(data, callback) {
  console.log("Traitement des données...");
  callback(data);
}

f = (result) => {
    console.log("Résultat :", result);
}

processData("Bonjour", f);
```

3- Les Bases du Langage JavaScript

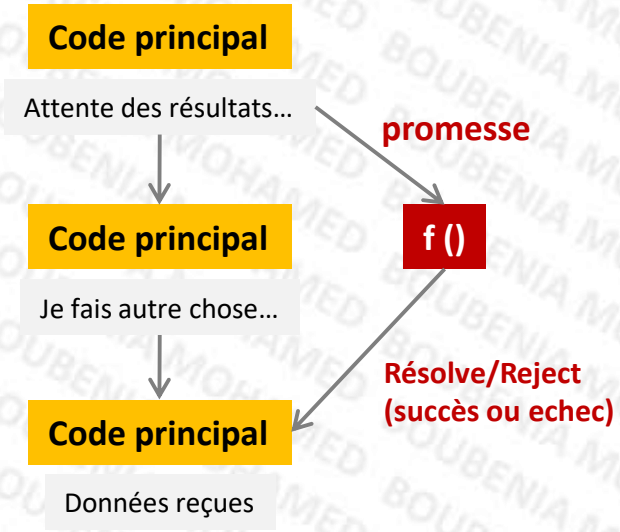
Une Promise est un objet qui représente une opération asynchrone.
C'est comme un contrat : « Je te promets que je te rendrai un résultat (ou une erreur), mais plus tard. »

```
// soit f() une fonction bloquante
f = (resolve, reject)=>{
    setTimeout(() => resolve("Données reçues !"), 1000);
}

// je peux l'exécuter de manière asynchrone non bloquante
const fetchData = new Promise((resolve, reject) => {
  console.log("Attente des résultat...");
  f(resolve, reject);
});

// je précise quoi faire quand elle se débloque
fetchData
  .then(result => console.log(result))
  .catch(error => console.error(error));

// je précise quoi faire en parallèle
console.log("Pendant que j'attends, je fais autre chose");
```



3- Les Bases du Langage JavaScript

Une façon plus lisible d'écrire un code asynchrone.

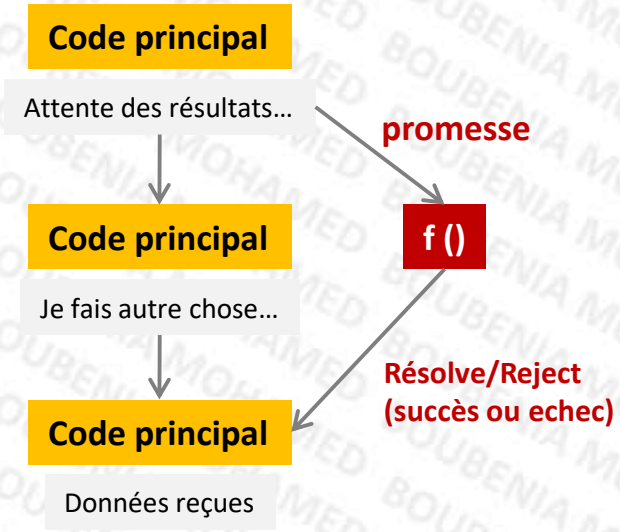
```
// soit f() une fonction bloquante
// Cette fois on met la promess dans f()
f = () => {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("Données reçues !"), 2000);
  });
};

async function fetchData() {
  try {
    console.log("Attente des résultat...");
    const result = await f(); // on appelle f() pour obtenir la Promise

    console.log(result);
  } catch (error) {
    console.error(error);
  }
}

fetchData();

// je précise quoi faire en parallèle
console.log("Pendant que j'attends, je fais autre chose");
```



3- Les Bases du Langage JavaScript


```
a = 'a'; // implicitement globale
b = 'b'; // implicitement globale

if(2>1){
    let d = 'd'; // let: portée dans le bloc courant
    e = 'e'; // implicitement globale
}

function myFunction(){
    c = 'c'; // implicitement globale
    let f= 'f'; // portée locale à la fonction
    const x= 'x';

     console.log("Appel depuis muFunction: "+a); // global
     console.log("Appel depuis muFunction: "+b); // global
     console.log("Appel depuis muFunction: "+c); // global
    // console.log("Appel depuis muFunction: "+d); // error
     console.log("Appel depuis muFunction: "+e); // global
     console.log("Appel depuis muFunction: "+f); // local
     console.log("Appel depuis muFunction: "+x); // local

}
const y = 'y';

myFunction();
console.log("Appel global: "+a); // global
console.log("Appel global: "+b); // global
console.log("Appel global: "+c); // global, seulement si appel à myFonction
//console.log("Appel global: "+d); // error: 'c' porté locale au block if
console.log("Appel global: "+e);
//console.log("Appel global: "+f); // error: 'f' porté locale à myFunction
//console.log("Appel global: "+x); // error: 'x' porté locale à myFunction
console.log("Appel global: "+y);
```

```
Appel depuis muFunction: a
Appel depuis muFunction: b
Appel depuis muFunction: c
Appel depuis muFunction: e
Appel depuis muFunction: f
Appel depuis muFunction: x
Appel global: a
Appel global: b
Appel global: c
Appel global: e
Appel global: y
```

3- Les Bases du Langage JavaScript

Initialiser une variable sans la préfixer par Let ou Const ou Var est une mauvaise pratique.
Elle aura une portée globale implicite, même dans un bloc.

```
x = 10;
```

La bonne pratique voudrait qu'on maitrise manuellement la portée de nos variable.

Pour ne pas se tromper, on utilise le mode stricte, qui nous signale les variables non définie avec Let, ou Const, ou Var:

```
"use strict";
x = 10;
```

```
Uncaught ReferenceError: x is not defined
    at test JS.html:10:6
```

Pour éviter les erreurs, on utilise donc let pour les variables, et const pour les constantes

```
"use strict";
let x = 10;
const pi = "3.14";
```

Dans les anciennes version de JS, on utilisait le mot clé var.

```
"use strict";

if(true){
    var x = 10;
}

console.log(x);
```

Mais le problème avec var c'est que la variable était Rehostée (hoisting), et donc sa portée devenait globale

Les bases du langage JavaScript

D'abord, Il est important de connaitre le langage JavaScript (syntaxe, sémantique, etc.)

Nous allons décortiquer

Script JS

```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation

HTML

```
<!DOCTYPE html>
<html>
    <head>
    </head>
    <body>
        <h1 id="compt_H1">Compteur=0</h1>
        <button onclick="doIncrement()">
            Incrémenter
        </button>
    </body>
</html>
```

Evénement

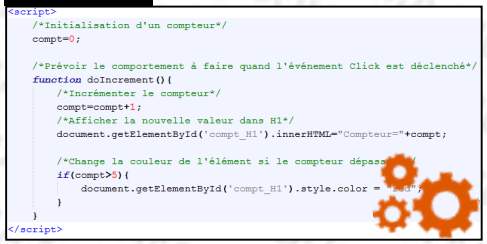
Quelque chose se passe: Plusieurs événements sont possibles, par exemple: onclick (au clic), onmouseover (au survol), onload (au chargement de la page), etc.

Visuel



Nous allons décortiquer

Script JS



```
<script>
    /*Initialisation d'un compteur*/
    compt=0;

    /*Prévoir le comportement à faire quand l'événement Click est déclenché*/
    function doIncrement(){
        /*Incrémenter le compteur*/
        compt=compt+1;
        /*Afficher la nouvelle valeur dans H1*/
        document.getElementById('compt_H1').innerHTML="Compteur="+compt;

        /*Change la couleur de l'élément si le compteur dépass
        if(compt>5){
            document.getElementById('compt_H1').style.color =
        }
    }
</script>
```

Traitement

Plusieurs types de traitements sont possibles, comme ce qui peut se faire dans un langage de programmation: déclaration de variables, opérations arithmétiques, affectations, comparaisons, tableaux, etc. En langage JavaScript.

Changement visuel

On sélectionne un élément HTML (presque comme le fait le CSS), puis on modifie le code HTML pour, par exemple, changer le contenu de l'élément.
Ou, on modifie le style, pour, par exemple, changer la couleur d'un élément.
Tout est modifiable par programmation.